

Department of Computer Science and Engineering

Deep Learning with NLP

Study Guide

Mr. HIMANSHU TIWARI
CSE Dept, PIET
Parul University

Unit 2: Introduction to Deep Learning

1. What is Deep Learning?

Deep Learning is a branch of Machine Learning that uses Artificial Neural Networks (ANNs) with multiple hidden layers to learn patterns and relationships from data automatically. It is inspired by the structure and working of the human brain. Deep learning systems can analyze large volumes of structured and unstructured data and perform complex tasks such as image recognition, speech recognition, natural language processing, and recommendation systems.

Unlike traditional programming, deep learning models do not require manual feature extraction. Instead, the system automatically learns useful features directly from data during training.

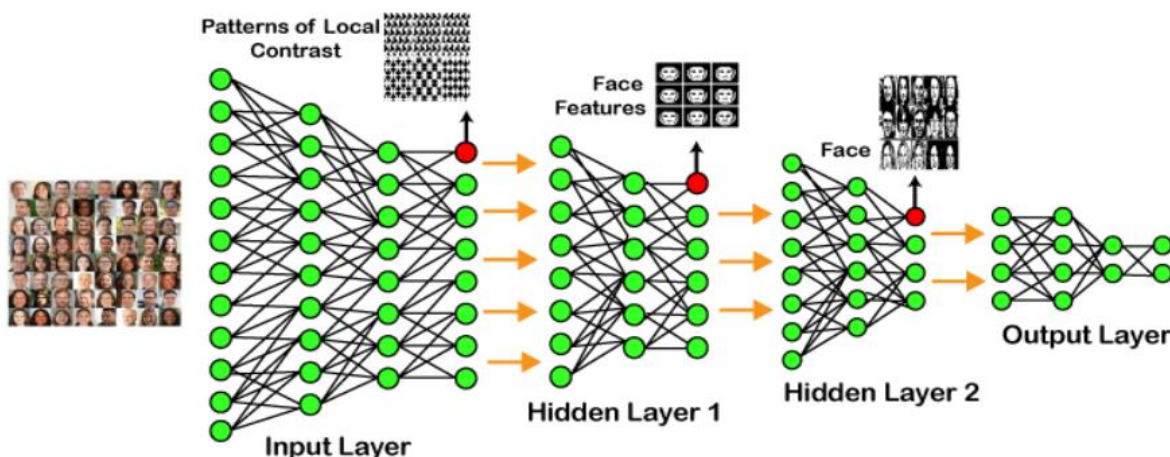


Fig 2.1. Example of Deep Learning

- In the provided example, the raw image data is fed into the initial input layer. This layer then analyzes the local contrast patterns, distinguishing between colors, luminosity, and other characteristics.
- Subsequently, the first hidden layer identifies facial features such as eyes, nose, and lips, focusing on these specific elements. The hidden layer then aligns these facial features with the appropriate face template.
- The second hidden layer plays a crucial role in determining the correct face, as demonstrated in the image above. The processed information is then passed to the output layer. To address more intricate tasks, additional hidden layers can be added.
- For instance, if the objective is to identify faces with specific attributes like a fair or dark complexion.

- By increasing the number of hidden layers, complex problems can be effectively tackled.

Features of Deep Learning

1. Uses multiple hidden layers for learning complex representations
2. Learns automatically from large datasets
3. Reduces the need for manual feature engineering
4. Capable of solving highly complex problems
5. Provides high accuracy in prediction and classification tasks

Applications of Deep Learning

1. Image and Video Recognition
2. Speech Recognition
3. Natural Language Processing
4. Autonomous Vehicles
5. Recommendation Systems
6. Fraud Detection
7. Healthcare Diagnosis
8. Game Playing AI
9. Time Series Forecasting
10. Machine Translation

Advantages of Deep Learning

1. High accuracy for complex tasks
2. Automatic feature extraction
3. Handles large and unstructured data efficiently
4. Continuous improvement with more data
5. Effective for image, text, and speech processing

Disadvantages of Deep Learning

1. Requires large amounts of labeled data
2. High computational requirements
3. Training takes significant time
4. Difficult to interpret results (Black Box Nature)
5. Requires powerful hardware such as GPUs

2.Types of Learning

Deep learning can be used in supervised learning, unsupervised learning, and reinforcement learning.

2.1 Supervised Learning

Supervised learning trains a neural network using labeled datasets. The model learns relationships between input features and target outputs.

Examples

6. Image Classification
7. Spam Detection
8. Sentiment Analysis
9. Speech Recognition

Advantages

10. High prediction accuracy
11. Easy performance evaluation
12. Suitable for classification and regression tasks

2.2 Unsupervised Learning

In unsupervised learning, the neural network identifies hidden patterns from unlabeled data.

Applications

13. Clustering
14. Dimensionality Reduction
15. Anomaly Detection
16. Customer Segmentation

Common Models

1. Autoencoders
2. Generative Models

2.3 Reinforcement Learning

Reinforcement learning trains an agent to make decisions by interacting with an environment. The agent receives rewards or penalties depending on its actions.

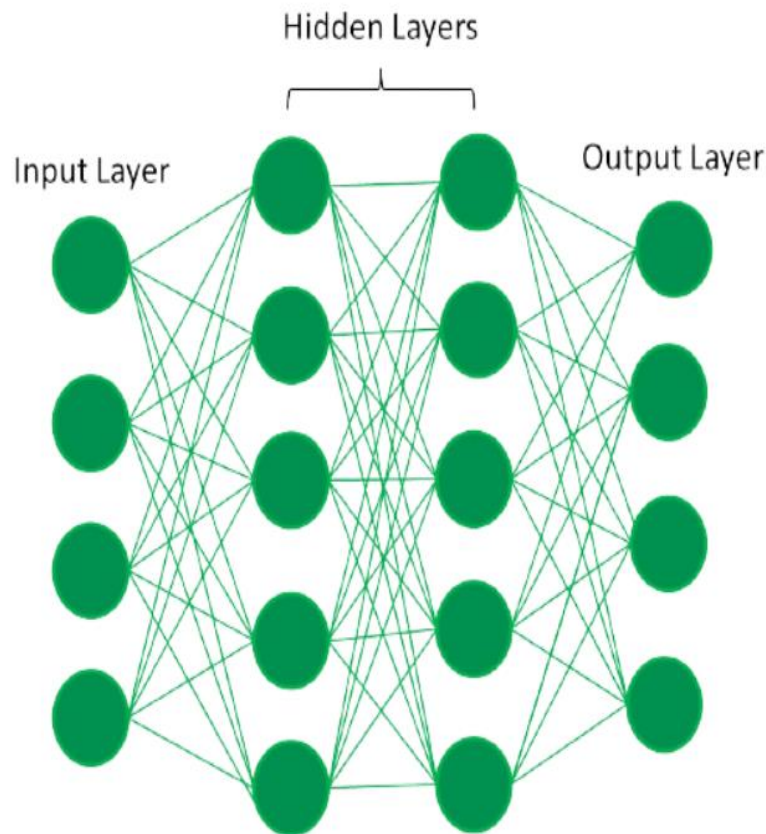
Applications

1. Robotics
2. Self-driving Cars
3. Game Playing AI
4. Recommendation Systems

Popular Algorithms

1. Deep Q-Networks (DQN)
2. Deep Deterministic Policy Gradient (DDPG)

Artificial Neural Networks (ANNs) are inspired by biological neurons. A neural network consists of interconnected neurons organized into layers.



Fully Connected Artificial Neural Network

Artificial neurons, referred to as units, are integral components of artificial neural networks. These networks consist of interconnected artificial neurons organized into layers.

The complexity of a neural network is determined by the intricate patterns present in the dataset, whether a layer contains a small number or millions of units.

Typically, an artificial neural network comprises an input layer, an output layer, and hidden layers. The input layer receives data from the external environment that the neural network aims to analyze or learn from.

Components of Neural Network

1. Input Layer

Receives input data from the external environment.

2. Hidden Layer

Processes information using weighted computations and activation functions.

3. Output Layer

Produces the final prediction or classification.

Working of Neural Network

1. Inputs are provided to the input layer.
2. Inputs are multiplied with weights.

3. Bias values are added.
4. Activation function is applied.
5. Output is forwarded to the next layer.
6. Final prediction is generated.

Important Terms

Weight

Represents the importance of an input.

Bias

Helps shift the activation function.

Neuron

A computational unit that processes inputs and produces outputs.

Net Input Equation

The net input of a neuron is calculated as:

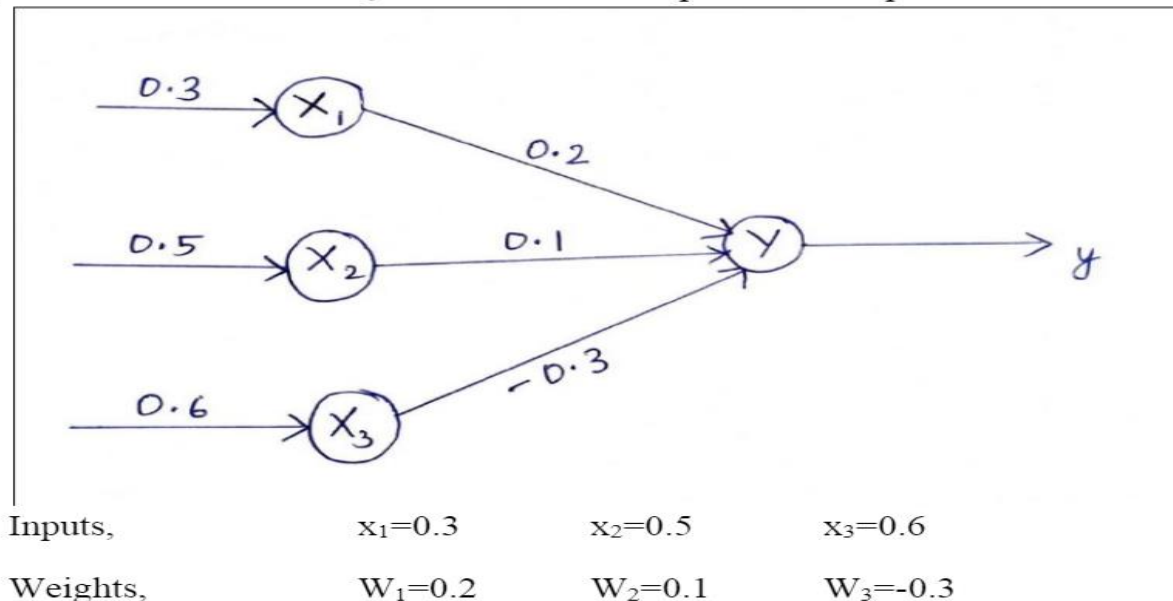
$$y_{in} = x_1 W_1 + x_2 W_2 + x_3 W_3$$

Where:

- x = input values
- w = weights
- b = bias

Example

(1) For the network shown, calculate the net input to the output neuron.



Net Input,

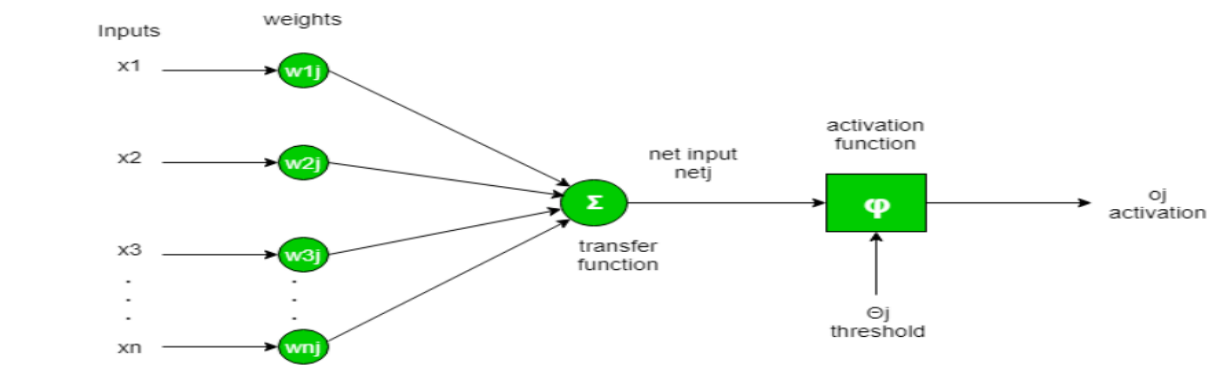
$$\begin{aligned}
 y_{in} &= x_1 W_1 + x_2 W_2 + x_3 W_3 \\
 &= 0.3 \cdot 0.2 + 0.5 \cdot 0.1 + 0.6 \cdot (-0.3) \\
 &= 0.06 + 0.05 - 0.18 \\
 &= 0.11 - 0.18 \\
 &= -0.07
 \end{aligned}$$

Difference between Machine Learning and Deep Learning

Machine Learning	Deep Learning
Uses statistical algorithms	Uses deep neural networks
Requires manual feature extraction	Learns features automatically
Works with smaller datasets	Requires large datasets
Faster training	Slower training
Easier to interpret	Works like a black box
Suitable for simple tasks	Suitable for complex tasks

3.What is an Activation Function?

Activation functions determine whether a neuron should be activated or not. They introduce non-linearity into neural networks.



Mathematically,

$$\text{net input} = \sum (\text{weight} * \text{input}) + \text{bias}$$

Importance of Activation Functions

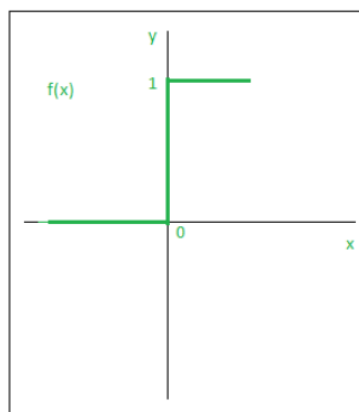
1. Enables learning of complex patterns
2. Controls neuron output
3. Helps neural networks solve non-linear problems

1. Step function

The Step Function activates the neuron if the net input exceeds a threshold value.

Mathematically, the Step Function can be represented as follows:

$$f(x) = 1, \text{ if } x \geq 0$$
$$f(x) = 0, \text{ if } x < 0$$

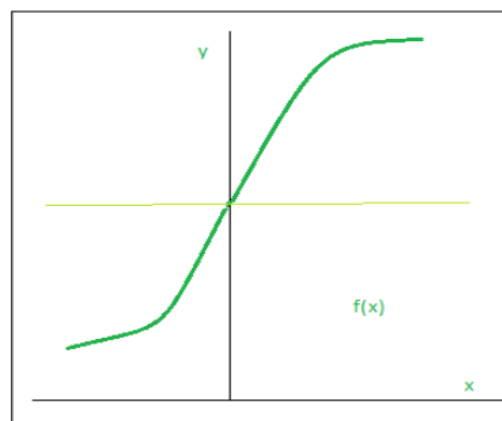


Characteristics

1. Simple implementation
2. Used in early neural networks
3. Cannot handle complex non-linear tasks effectively

2. Sigmoid function is a widely used activation function. It is defined as:

- The sigmoid function is a smooth and continuously differentiable function. Its key advantage over the step and linear functions is its non-linearity, which makes it an intriguing feature.
- When multiple neurons adopt the sigmoid function as their activation function, the resulting output is also non-linear. The sigmoid function has a distinct S-shaped curve and maps input values to a range between 0 and 1.



The sigmoid function maps input values between 0 and 1.

Advantages

1. Smooth and differentiable
2. Useful for binary classification
3. Produces probabilistic outputs

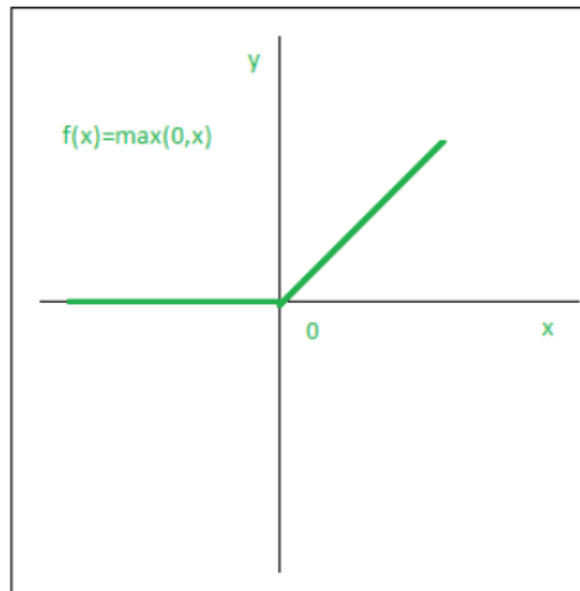
Disadvantages

1. Vanishing gradient problem
2. Computationally expensive

3. ReLU:

The ReLU function is the Rectified linear unit. It is the most widely used activation function. It is defined as:

- The primary benefit of employing the Rectified Linear Unit (ReLU) function as an activation function is that it does not activate all neurons simultaneously. In practical terms, this means that when the input to the ReLU function is negative, it is converted to zero, resulting in the neuron remaining inactive or "not activated."



ReLU stands for Rectified Linear Unit.

Advantages

1. Fast computation
2. Reduces vanishing gradient problem
3. Widely used in deep learning

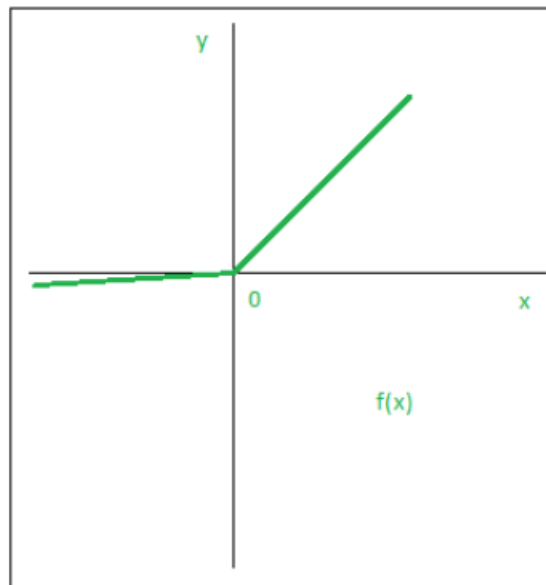
Disadvantages

1. Dying ReLU problem

4. LeakyReLU:

Leaky ReLU function is nothing but an improved version of the ReLU function. Instead of defining the ReLU function as 0 for x less than 0, we define it as a small linear component of x . It can be defined as:

$$\begin{aligned} f(x) &= ax, x < 0 \\ f(x) &= x, \text{ otherwise} \end{aligned}$$



Leaky ReLU is an improved version of ReLU.

Advantages

1. Prevents dying ReLU problem
2. Improves gradient flow

4.Types of neural networks

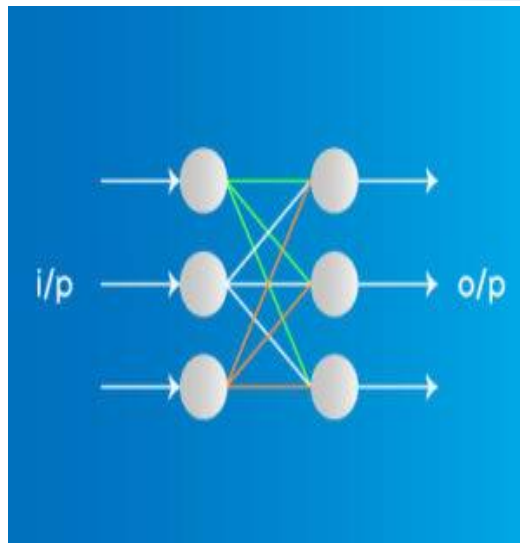
Deep learning includes several neural network architectures.

Major Types

1. Feedforward Neural Network (FNN)
2. Convolutional Neural Network (CNN)
3. Recurrent Neural Network (RNN)
4. Encoder Decoder Networks
5. Recursive Neural Networks
6. Multilayer Neural Networks

4.1 Feedforward Neural Network (FNN)

A Feedforward Neural Network is the simplest type of neural network where information flows only in one direction, from input layer to output layer.



feedforward neural network

Characteristics

1. Unidirectional data flow
2. No feedback connections
3. Easy to implement
4. Suitable for simple classification tasks

Applications

1. Pattern Recognition
2. Face Recognition
3. Speech Recognition
4. Character Recognition

Advantages

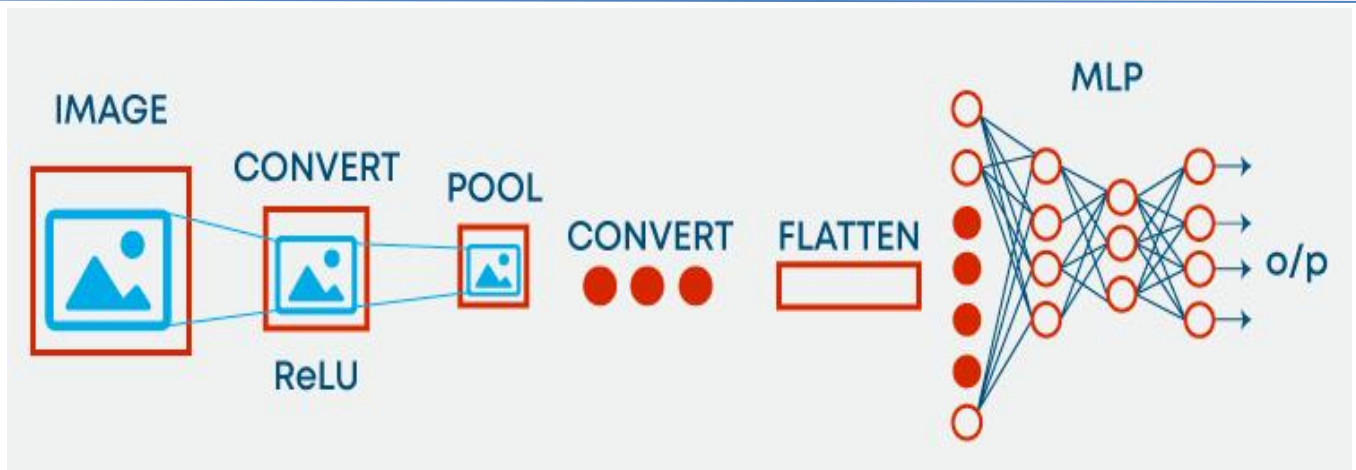
1. Simple architecture
2. Fast computation
3. Easy maintenance

Disadvantages

1. Cannot handle sequential data
2. Limited learning capability

4.2 Convolutional Neural Network

CNN is a deep learning architecture mainly used for image processing and computer vision tasks.



Convolutional neural network

Important Layers in CNN

1. Convolution Layer

Extracts important features from images using filters.

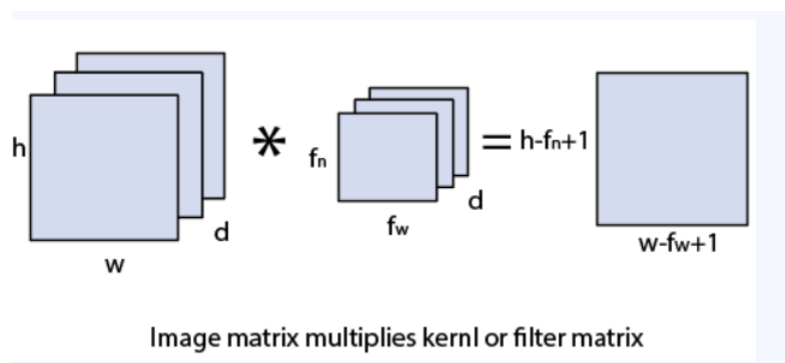
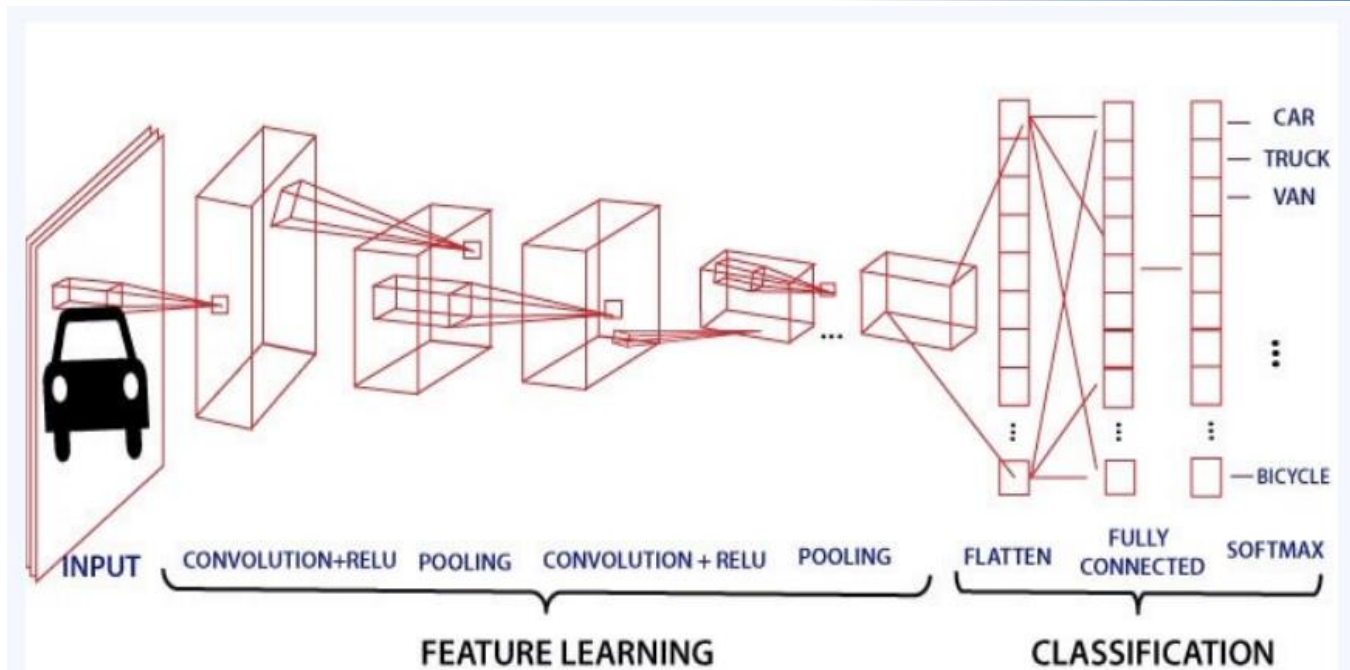
2. Pooling Layer

Reduces image size and computational complexity.

3. Fully Connected Layer

Performs classification.

Convolution Operation



CNN applies filters over images to extract patterns.

Output Size Formula : The convolution of the image with different filter values can produce a blur or sharpened image.

The size of the output image is calculated by:

$$i - k + 1 = 5 - 3 + 1 = 3$$

$$\begin{bmatrix} 1 & 1 & 1 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 & 0 \\ 0 & 1 & 1 & 0 & 0 \end{bmatrix} \times \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 1 \end{bmatrix}$$

Where:

- I = Input Size
- K = Kernel Size

5 × 5 – Image Matrix 3 × 3 – Filter Matrix

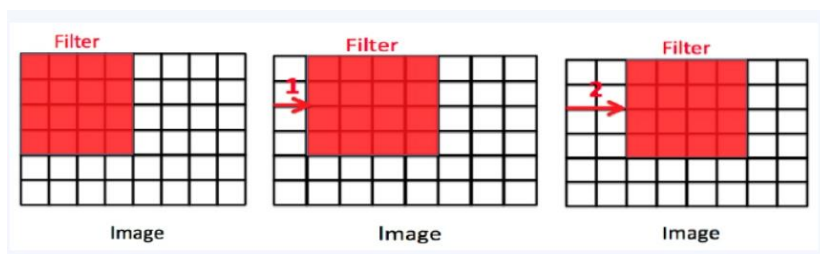
1	1	1	0	0
0	1	1	1	0
0	0	1 _{x1}	1 _{x0}	1 _{x1}
0	0	1 _{x0}	1 _{x1}	0 _{x0}
0	1	1 _{x1}	0 _{x0}	0 _{x1}

4	3	4
2	4	3
2	3	4

Stride

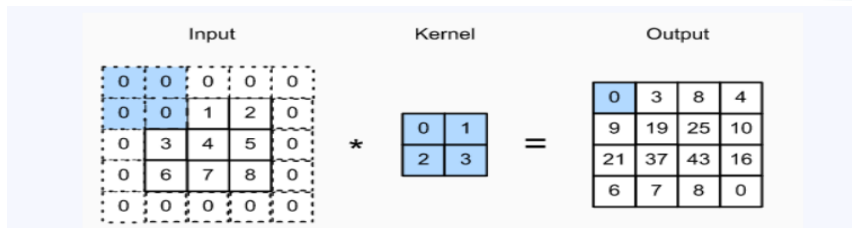
- Stride determines how many pixels the filter moves during convolution. When the array is created, the pixels are shifted over to the input matrix. The number of pixels turning to the input matrix is known as the strides.
- When the number of strides is 1, we move the filters to 1 pixel at a time.
- Similarly, when the number of strides is 2, we carry the filters to 2 pixels, and so on.
- They are essential because they control the convolution of the filter against the input, i.e., Strides are responsible for regulating the features that could be missed while flattening the image.
- They denote the number of steps we are moving in each convolution. The following figure shows how the convolution would work.

$$O = \lceil \frac{i-k}{s} \rceil + 1 = \lceil \frac{5-3}{2} \rceil + 1 = 2$$



Padding

- Padding preserves image size after convolution. The padding plays a vital role in creating CNN. After the convolution operation, the original size of the image is shrunk. Also, in the image classification task, there are multiple convolution layers after which our original image is shrunk after every step, which we don't want.
- When the kernel moves over the original image, it passes through the middle layer more times than the edge layers, due to which there occurs an overlap.
- To overcome this problem, a new concept was introduced named padding. It is an additional layer that can add to the borders of an image while preserving the size of the original picture. For example:



Types of Pooling

Pooling:

The pooling layer is another building block of a CNN and plays a vital role in pre-processing an image.

In the pre-process, the image size shrinks by reducing the number of parameters if the image is too large.

When the picture is shrunk, the pixel density is also reduced, the downscaled image is obtained from the previous layers.

Basically, its function is to progressively reduce the spatial size of the image to reduce the network complexity and computational cost.

Spatial pooling is also known as downsampling or subsampling that reduces the dimensionality of each map but retains the essential features.

A rectified linear activation function, or ReLU, is applied to each value in the feature map.

Relu is a simple and effective nonlinearity that does not change the values in the feature map but is present because later subsequent pooling layers are added. Pooling is added after the nonlinearity is applied to the feature maps.

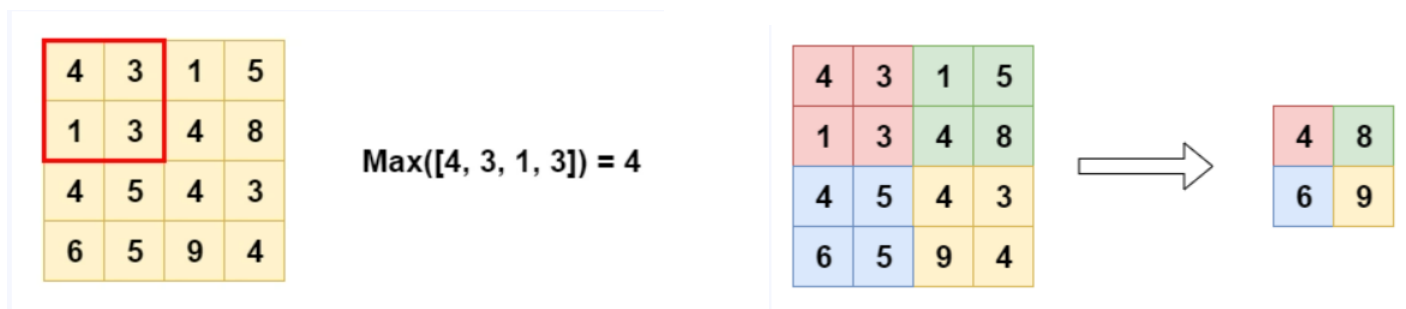
Max Pooling

Max pooling is a rule to take the maximum of a region and help to proceed with the most crucial features from the image.

It is a sample-based process that transfers continuous functions into discrete counterparts.

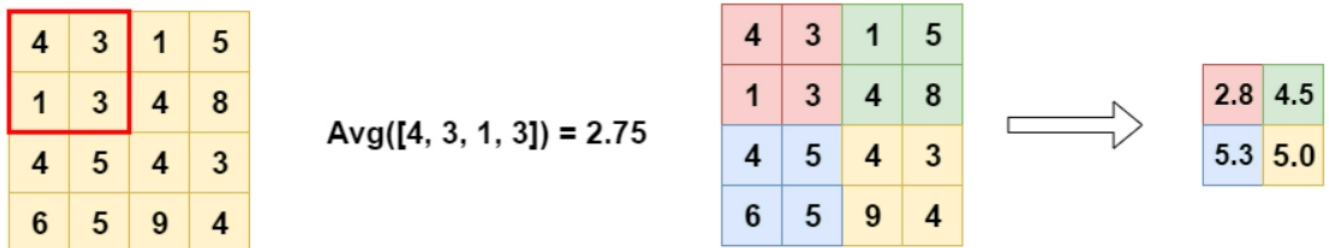
Its primary objective is to downscale an input by reducing its dimensionality and making assumptions about features contained in the sub-region that were rejected.

Selects maximum value from a region.



Average Pooling

- It retains information about the lesser essential features.
- It simply downscales by dividing the input matrix into rectangular regions and calculating the average values of each area.



Computes average value from a region.

Sum Pooling

Calculates sum of elements in a region.

Applications of CNN

1. Image Classification
2. Medical Image Analysis
3. Face Detection
4. Autonomous Vehicles
5. Video Processing

Advantages

1. High accuracy in image processing
2. Automatic feature extraction
3. Reduces preprocessing requirements

Disadvantages

1. Requires high computational resources
2. Needs large datasets

4.3 Recurrent Neural Network

RNN is designed for sequential and temporal data processing.

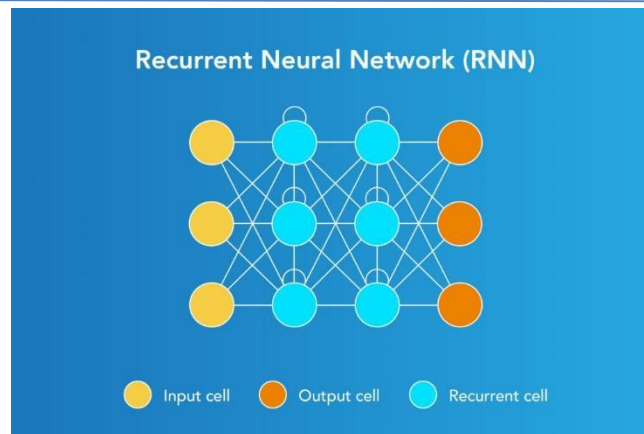


Figure 4.3.1 Recurrent Neural Networks

Working Principle

1. Processes input sequentially
2. Maintains hidden memory state
3. Uses previous outputs for future predictions

Applications

1. Language Translation
2. Text Prediction
3. Sentiment Analysis
4. Speech Recognition

Advantages

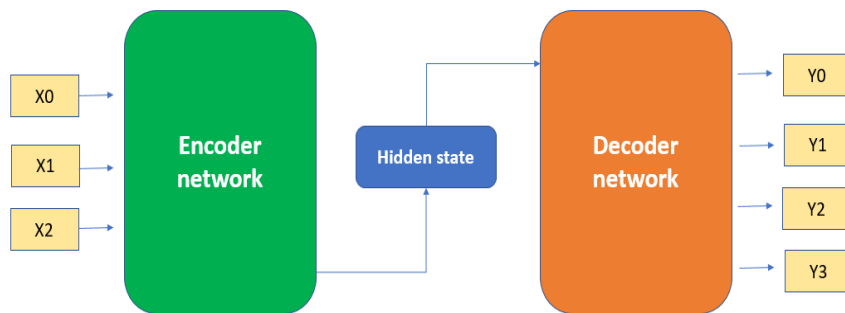
1. Handles sequential data effectively
2. Maintains contextual memory
3. Useful for language processing tasks

Disadvantages

1. Vanishing gradient problem
2. Slow training process
3. Difficulty handling long sequences

4.4 Encoder Decoder

Encoder-Decoder architecture converts an input sequence into an output sequence.



4.4.1. Encoder Decoder Neural Networks

Components

Encoder

Converts input sequence into latent representation.

Decoder

Generates output sequence using encoded information.

Applications

1. Machine Translation
2. Chatbots
3. Text Summarization
4. Speech Recognition

Advantages

1. Handles variable-length sequences
2. Effective for NLP tasks

Limitations

1. Requires large training data
2. Computationally expensive

4.5 Recursive Neural Network

- Recursive Neural Networks (RvNNs) are a class of deep neural networks that can learn detailed and structured information.
- The word recursive indicates that the neural network is applied to its output.
- Recursive Neural Networks (RvNNs) are a type of deep neural network architecture that can effectively process and analyze hierarchical or tree-like structured data. They are

particularly useful for tasks such as parsing sentences or working with tree-structured inputs.

- The key idea behind RvNNs is the recursive application of the same set of weights on structured inputs.
- This recursive process involves combining child nodes to generate parent nodes in a hierarchical manner.
- Each child-parent connection in the tree structure is associated with a weight matrix, and similar child nodes share the same weights. This shared weight scheme allows the RvNN to leverage the structural information present in the data.

Recursive Neural Networks process hierarchical and tree-structured data.

Characteristics

1. Uses recursive architecture
2. Learns hierarchical relationships
3. Useful for syntactic parsing

Applications

1. Sentiment Analysis
2. Sentence Parsing
3. Semantic Analysis

Advantages

1. Handles hierarchical structures efficiently
2. Captures long-term dependencies

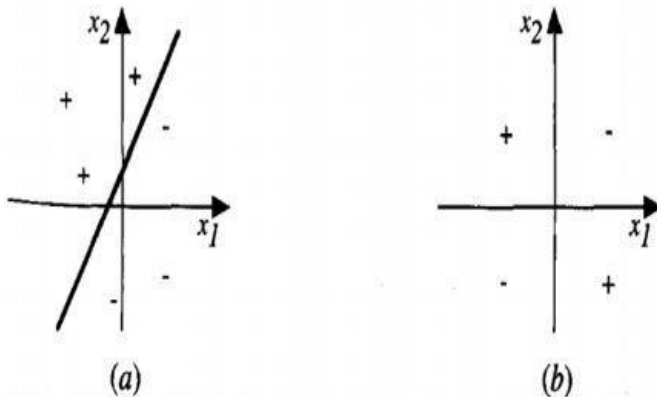
Disadvantages

1. Complex implementation
2. Slow parsing process
3. High computational cost

4.6 Multilayer Neural Networks

Multilayer Neural Networks contain one or more hidden layers.

- A single-layer neural network will work only for linearly separable data and not for non-linearly separable data. Hence there is a need for Multilayer Neural Networks, to be able to work with non-linearly separable data.
- A multi-layer Neural Network has two hidden layers. Hidden layers, whose neurons are not directly linked to the output, are used in multilayer networks to address the classification issue for non-linear data.
- The hidden layers can be understood geometrically as extra hyper-planes that increase the network's separation capability. Typical multilayer network designs are seen in the Figure 6.6.1& 2.



4.6.1 Figure (a) -> Training Set is Linearly Separable

4.6.2. Figure (b) -> Training Set is non-linearly Separable

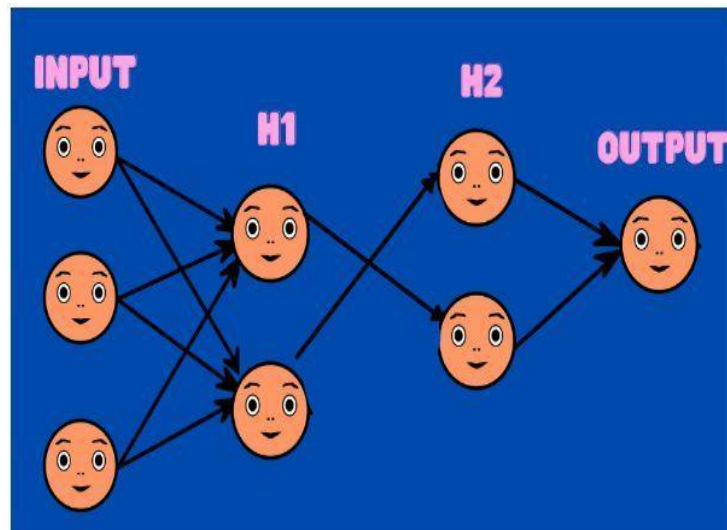


Figure 4.6.3. Multilayer Neural Networks

Importance

Single-layer neural networks can solve only linearly separable problems. Multilayer networks solve non-linearly separable problems.

Advantages

1. Solves complex problems
2. Better learning capability
3. Improved classification accuracy

Disadvantages

1. Increased computational complexity

2. Longer training time

5. Comparison Between CNN and RNN

CNN	RNN
Used for spatial data	Used for sequential data
Suitable for image processing	Suitable for text and speech
Fixed input/output size	Variable input/output size
Faster training	Slower training
Uses convolution operations	Uses recurrent memory

6. Stochastic Gradient Descent

What is Gradient Descent?

- Gradient Descent is an optimization algorithm used to minimize the cost function by updating model parameters. Gradient Descent is a widely employed technique in machine learning projects, involving an iterative optimization process aimed at locating the optimal value (either minimum or maximum) of an objective function. This method effectively adjusts a model's parameters with the goal of minimizing a cost function.
- The main objective of gradient descent is to determine the model parameters that yield optimal accuracy on both training and test datasets.
- In gradient descent, the gradient refers to a vector that indicates the general direction of the steepest ascent of the function at a specific point.
- By moving in the opposite direction of the gradient, the algorithm gradually descends towards lower values of the function, ultimately reaching its minimum.
- Stochastic Gradient Descent (SGD) differs from traditional Gradient Descent by selecting only a single random training example or a small batch for each iteration to compute the gradient and update the model parameters. This random selection introduces randomness into the optimization process, which is why it is referred to as "stochastic" Gradient Descent.

Stochastic Gradient Descent

- Stochastic Gradient Descent updates model parameters using one training sample at a time. The main advantage of SGD is its **computational efficiency**, particularly when working with large datasets. By utilizing a single example or a small batch, the computational cost per iteration is greatly reduced compared to conventional Gradient Descent methods that necessitate processing the entire dataset.

- **Stochastic Gradient Descent Algorithm:**
- **Initialization:** Randomly assign initial values to the model's parameters.
- **Set Parameters:** Determine the number of iterations and the learning rate (α) for parameter updates.
- **Stochastic Gradient Descent Loop:** Repeat the following steps until either the model converges or the maximum number of iterations is reached:
 - a. Introduce randomness by shuffling the training dataset.
 - b. Iterate over each training example (or a small batch) in the shuffled order.
 - c. Calculate the gradient of the cost function with respect to the model parameters using the current training example (or batch).
 - d. d. Update the model parameters by taking a step in the direction opposite to the gradient, scaled by the learning rate.
 - e. e. Evaluate the convergence criteria, such as comparing the cost function between consecutive gradient iterations. Return Optimized Parameters: Once the convergence criteria are satisfied or the maximum iterations are reached, return the optimized model parameters.
 - f. **In SGD, due to selecting only one random sample from the dataset per iteration, the algorithm's path to the minimum is typically noisier compared to standard Gradient Descent. However, the precise path taken by the algorithm is less significant as long as it reaches the minimum, and it achieves this goal with significantly reduced training time.**

Working Steps of SGD

1. Initialize model parameters randomly
2. Select learning rate
3. Shuffle training data
4. Compute gradient for one sample
5. Update parameters
6. Repeat until convergence

Advantages of SGD

1. Faster training
2. Memory efficient
3. Escapes local minima

Disadvantages of SGD

1. Noisy updates
2. Slow convergence
3. Sensitive to learning rate

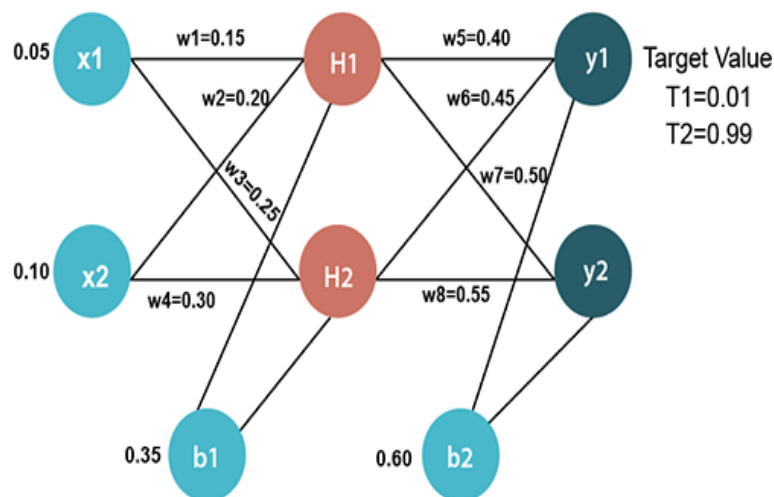
7. Backpropagation Algorithm

What is Backpropagation?

- Backpropagation is a supervised learning algorithm used to train neural networks by minimizing error through gradient descent. Backpropagation, an essential concept in neural networks, plays a crucial role in optimizing data classification.
- To achieve this, we need to update the weights of the parameters and biases. However, performing this task in a deep neural network poses a challenge. In linear regression, we employ gradient descent to optimize the parameters. Similarly, in deep neural networks, we utilize the Backpropagation algorithm along with gradient descent.
- For a single training example, the Backpropagation algorithm computes the gradient of the error function. It can be expressed as a function within the neural network. Backpropagation encompasses a collection of methods that efficiently train artificial neural networks by leveraging a gradient descent approach, which makes use of the chain rule.
- The key characteristics of Backpropagation are its iterative, recursive, and efficient nature. It calculates updated weights to enhance the network's performance until it can successfully execute the designated task it is being trained for. However, it is necessary to have knowledge of the derivatives of the activation function during the network design phase for Backpropagation to work effectively.

Working Principle

1. Perform forward propagation
 2. Compute output error
 3. Propagate error backward
 4. Update weights and biases
 5. Repeat until error reduces
- **Forward Pass Example: To gain a deeper understanding of how Backpropagation updates weights, let's begin with an illustrative example and examine the mathematical process involved.**



Input values

X1=0.05 X2=0.10

Initial weight

W1=0.15
W2=0.20
W3=0.25
W4=0.30

w5=0.40
w6=0.45
w7=0.50
w8=0.55

Bias Values

b1=0.35 b2=0.60

Target Values

T1=0.01 T2=0.99

Now, we first calculate the values of H1 and H2 by a forward pass.

Forward Pass

To find the value of H1 we first multiply the input value from the weights as

$$H1 = x1 \times w1 + x2 \times w2 + b1$$

$$H1 = 0.05 \times 0.15 + 0.10 \times 0.20 + 0.35$$

$$H1 = 0.3775$$

To calculate the final result of H1, we performed the

$$H1_{final} = \frac{1}{1 + \frac{1}{e^{H1}}}$$

$$H1_{final} = \frac{1}{1 + \frac{1}{e^{0.3775}}}$$

$$H1_{final} = 0.593269992$$

We will calculate the value of H2 in the same way as H1

$$H2 = x1 \times w3 + x2 \times w4 + b1$$

$$H2=0.05 \times 0.25 + 0.10 \times 0.30 + 0.35$$

$$H2=0.3925$$

To calculate the final result of H1, we performed the sigmoid function as

$$y1_{final} = \frac{1}{1 + \frac{1}{e^{y1}}}$$

$$y1_{final} = \frac{1}{1 + \frac{1}{e^{1.10590597}}}$$

$$y1_{final} = 0.75136507$$

We will calculate the value of y2 in the same way as y1

$$y2=H1 \times w7 + H2 \times w8 + b2$$

$$y2=0.593269992 \times 0.50 + 0.596884378 \times 0.55 + 0.60$$

$$y2=1.2249214$$

To calculate the final result of H1, we performed the sigmoid function as

$$y2_{final} = \frac{1}{1 + \frac{1}{e^{y2}}}$$

$$y2_{final} = \frac{1}{1 + \frac{1}{e^{1.2249214}}}$$

$$y2_{final} = 0.772928465$$

Our target values are 0.01 and 0.99. Our y1 and y2 value is not matched with our target values T1 and T2.

Now, we will find the total error, which is simply the difference between the outputs from

the target outputs. The total error is calculated as

$$E_{total} = \sum \frac{1}{2} (\text{target} - \text{output})^2$$

$$= \frac{1}{2} (t1 - y1_{final})^2 + \frac{1}{2} (T2 - y2_{final})^2$$

$$= \frac{1}{2} (0.01 - 0.75136507)^2 + \frac{1}{2} (0.99 - 0.772928465)^2$$

$$= 0.274811084 + 0.0235600257$$

$$E_{total} = 0.29837111$$

Advantages of Backpropagation

1. Improves network accuracy
2. Efficient weight optimization
3. Widely used in deep learning

Disadvantages

1. Computationally expensive
 2. Slow training for deep networks
 3. Requires differentiable activation functions
-

8. Basic Terms

Epoch

One complete cycle through the entire training dataset.

Iteration

One update of model parameters using a batch.

Batch

Subset of training data used in one iteration.